Pattern Extraction

Pattern Extraction Method

Pattern Extraction Method は、Knowledge Graph において
複数の case から共通する進行構造（pattern）を抽出する方法である。

Pattern は直接観察できるものではなく、
case の比較から発見される構造である。

そのため Pattern Extraction は

- case analysis
- case comparison
- abstraction

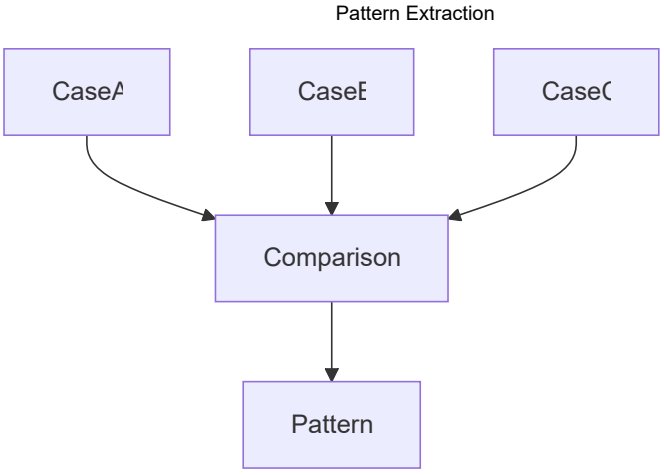
のプロセスを含む。

Pattern Extraction の目的

Pattern Extraction は次を目的とする。

- case の共通構造を発見する
- 知識を抽象化する
- mechanism 推論の基盤を作る

Pattern Extraction の基本構造



Pattern Extraction 手順

Step1

case を収集する。

最低

```
3 case
```

以上が望ましい。

Step2

case の構造を整理する。

各 case について

- condition
- trigger
- actor
- interaction
- outcome

を書く。

Step3

case comparison を行う。

共通点を探す。

Step4

進行構造を抽象化する。

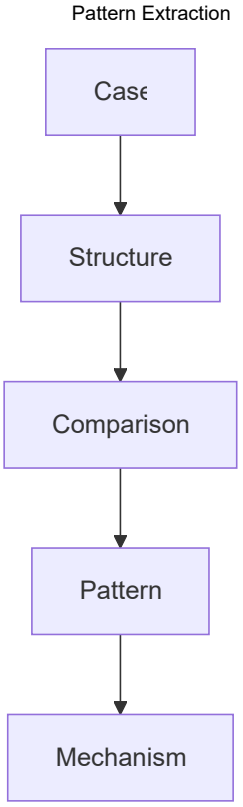
```
共通 sequence
```

を抽出する。

Step5

pattern ノートを作る。

Pattern Extraction の図



Pattern Extraction の例（抽象）

例

```
企業炎上
政治スキャンダル
SNSキャンセル
```

共通構造

```
規範逸脱
  ↓
情報拡散
  ↓
集団反応
  ↓
評判制裁
```

Pattern

評判制裁パターン

Pattern Extraction の分析軸

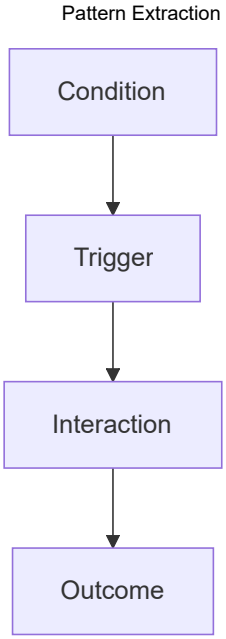
Pattern Extraction は次の軸で分析する。

軸	説明
condition	初期状態
trigger	発生原因
actor	主体
interaction	相互作用
outcome	結果

Pattern Extraction フレーム

```
condition
 ↓
trigger
 ↓
interaction
 ↓
outcome
```

Pattern Extraction 図



Pattern Extraction の注意

---

1 case が少ない

pattern が不安定になる。

---

2 特殊事例

共通構造が歪む。

---

3 outcome だけ抽象

進行構造が消える。

---

4 mechanism と混同

因果説明と構造を区別する。

Pattern Extraction と Mechanism

Pattern は mechanism の表れ。

```
pattern
 ↓
mechanism
```

Mechanism は
pattern の原因を説明する。

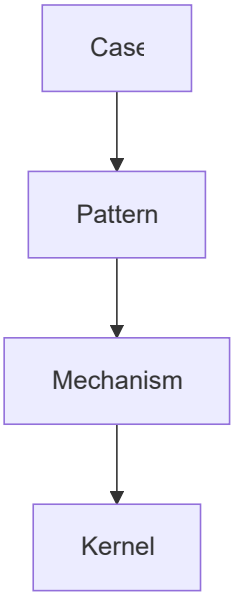
Pattern Extraction と Knowledge Graph

Pattern Extraction は

```
case layer
 ↓
pattern layer
```

への抽象化である。

Pattern Extraction の図



LLM にとっての意味

Pattern Extraction があると
LLM は

- case の共通構造を見つける
- 新しい pattern を発見する
- reasoning を一般化する

関連ノート

- 99_oldzettelkasten/04_knowledge_graph/Pattern
- 99_oldzettelkasten/04_knowledge_graph/Pattern Hub
- Pattern Comparison
- Case Comparison Method
- 99_oldzettelkasten/04_knowledge_graph/Mechanism

Pattern Extraction Method

Pattern Extraction Method

Pattern Extraction Method は、Knowledge Graph において
複数の case から共通する進行構造（pattern）を抽出する方法である。

Pattern は直接観察できるものではなく、
case の比較から発見される構造である。

そのため Pattern Extraction は

- case analysis
- case comparison
- abstraction

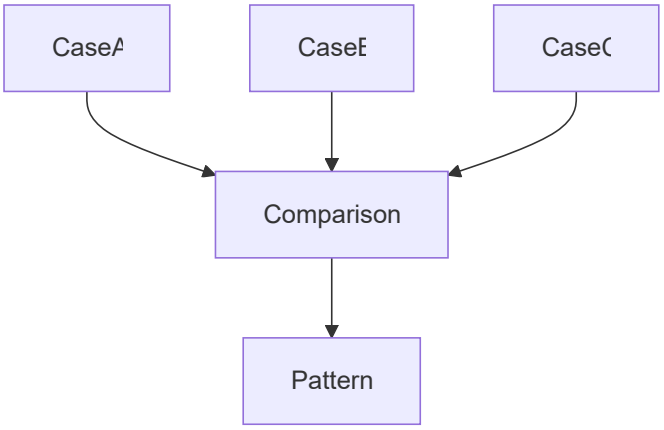
のプロセスを含む。

Pattern Extraction の目的

Pattern Extraction は次を目的とする。

- case の共通構造を発見する
- 知識を抽象化する
- mechanism 推論の基盤を作る

Pattern Extraction の基本構造



Pattern Extraction 手順

Step1

case を収集する。

最低

```
3 case
```

以上が望ましい。

Step2

case の構造を整理する。

各 case について

- condition
- trigger
- actor
- interaction
- outcome

を書く。

Step3

case comparison を行う。

共通点を探す。

Step4

進行構造を抽象化する。

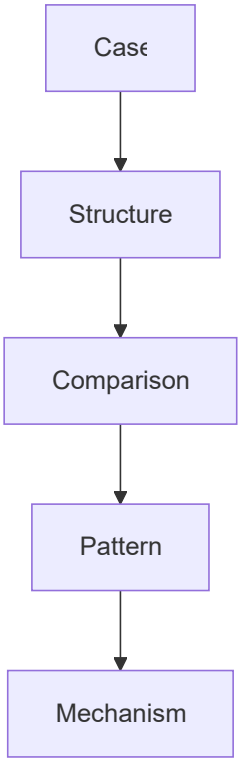
```
共通 sequence
```

を抽出する。

Step5

pattern ノートを作る。

Pattern Extraction の図



Pattern Extraction の例（抽象）

例

```
企業炎上
政治スキャンダル
SNSキャンセル
```

共通構造

```
規範逸脱
 ↓
情報拡散
 ↓
集団反応
 ↓
評判制裁
```

Pattern

```
評判制裁パターン
```

Pattern Extraction の分析軸

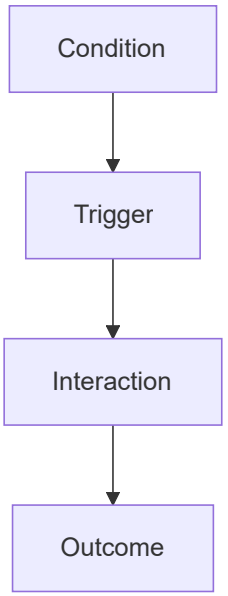
Pattern Extraction は次の軸で分析する。

軸	説明
condition	初期状態
trigger	発生原因
actor	主体
interaction	相互作用
outcome	結果

Pattern Extraction フレーム

```
condition
 ↓
trigger
 ↓
interaction
 ↓
outcome
```

Pattern Extraction 図



Pattern Extraction の注意

---

1 case が少ない

pattern が不安定になる。

---

2 特殊事例

共通構造が歪む。

---

3 outcome だけ抽象

進行構造が消える。

---

4 mechanism と混同

因果説明と構造を区別する。

Pattern Extraction と Mechanism

Pattern は mechanism の表れ。

```
pattern
 ↓
mechanism
```

Mechanism は
pattern の原因を説明する。

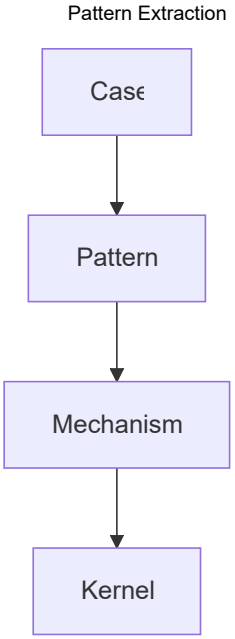
Pattern Extraction と Knowledge Graph

Pattern Extraction は

```
case layer
 ↓
pattern layer
```

への抽象化である。

Pattern Extraction の図



LLM にとっての意味

Pattern Extraction があると
LLM は

- case の共通構造を見つける
- 新しい pattern を発見する
- reasoning を一般化する

関連ノート

- 99_oldzettelkasten/04_knowledge_graph/Pattern
- 99_oldzettelkasten/04_knowledge_graph/Pattern Hub
- Pattern Comparison
- Case Comparison Method
- 99_oldzettelkasten/04_knowledge_graph/Mechanism

Pattern Boundary Rule

Pattern Boundary Rule

Pattern Boundary Rule は、Knowledge Graph において
pattern 同士の境界を明確に保つための設計ルールである。

Pattern は本来、現実に繰り返し現れる構造や進行を抽象化したものである。
しかし pattern を増やしていくと、次の問題が必ず発生する。

- 似た pattern が増える
- 同じ case が複数 pattern に当てはまる
- pattern の定義が曖昧になる
- pattern と mechanism が混ざる

Pattern Boundary Rule は、これを防ぐ。

Pattern 境界が崩れる理由

Pattern 境界が曖昧になるのは、主に次の理由による。

1. 抽象レベルの違い

例

- 炎上パターン
- 規範逸脱可視化パターン
- 評判崩壊パターン

これらは同じ現象を違う抽象度で見ている可能性がある。

---

2. mechanism と混同

例

- 同調パターン
- 社会的証明

これは実際には pattern と mechanism が混ざっている。

---

3. outcome を pattern としてしまう

例

- 崩壊パターン
- 成功パターン

結果だけを pattern にしてしまうと境界が不明瞭になる。

---

4. role を pattern にしてしまう

例

- リーダーパターン
- 裏切り者パターン

これは pattern ではなく role である可能性が高い。

---

5. case を pattern に昇格してしまう

例

- 〇〇炎上パターン
- △△崩壊パターン

特定事件に依存するものは pattern ではなく case である。

Pattern Boundary の基本原則

Pattern の境界を安定させるには、次の原則を守る。

原則1

pattern は **時間構造** を持つ。

pattern は

- 状態
- 変化

- 結果

の流れを含む。

つまり pattern は

```
状態 → 行動 → 反応 → 結果
```

という **進行構造** を持つ。

原則2

pattern は **複数 case に現れる**。

1つの事例だけなら pattern ではない。

原則3

pattern は **mechanism と区別する**。

mechanism は

「なぜ起きるか」

pattern は

「どういう形で起きるか」

である。

原則4

pattern は **role を含めるが role ではない**。

pattern は多くの場合、

- actor
- role
- interaction

を含むが、

role 単体は pattern ではない。

原則5

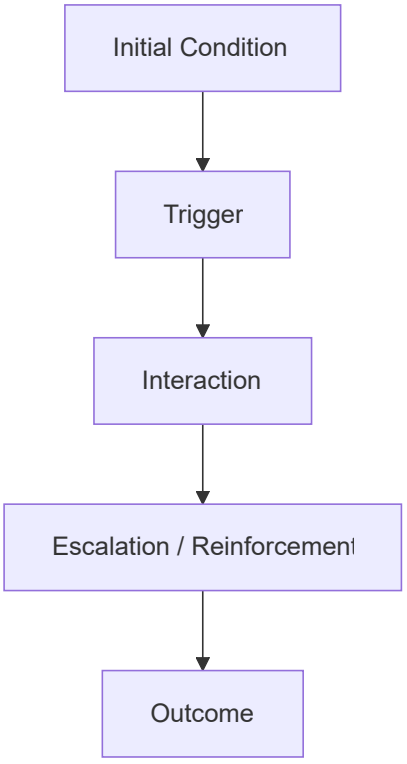
pattern は **結果ではない**

結果は outcome である。

pattern はその前の進行である。

Pattern の基本構造

pattern は一般に次の構造を持つ。



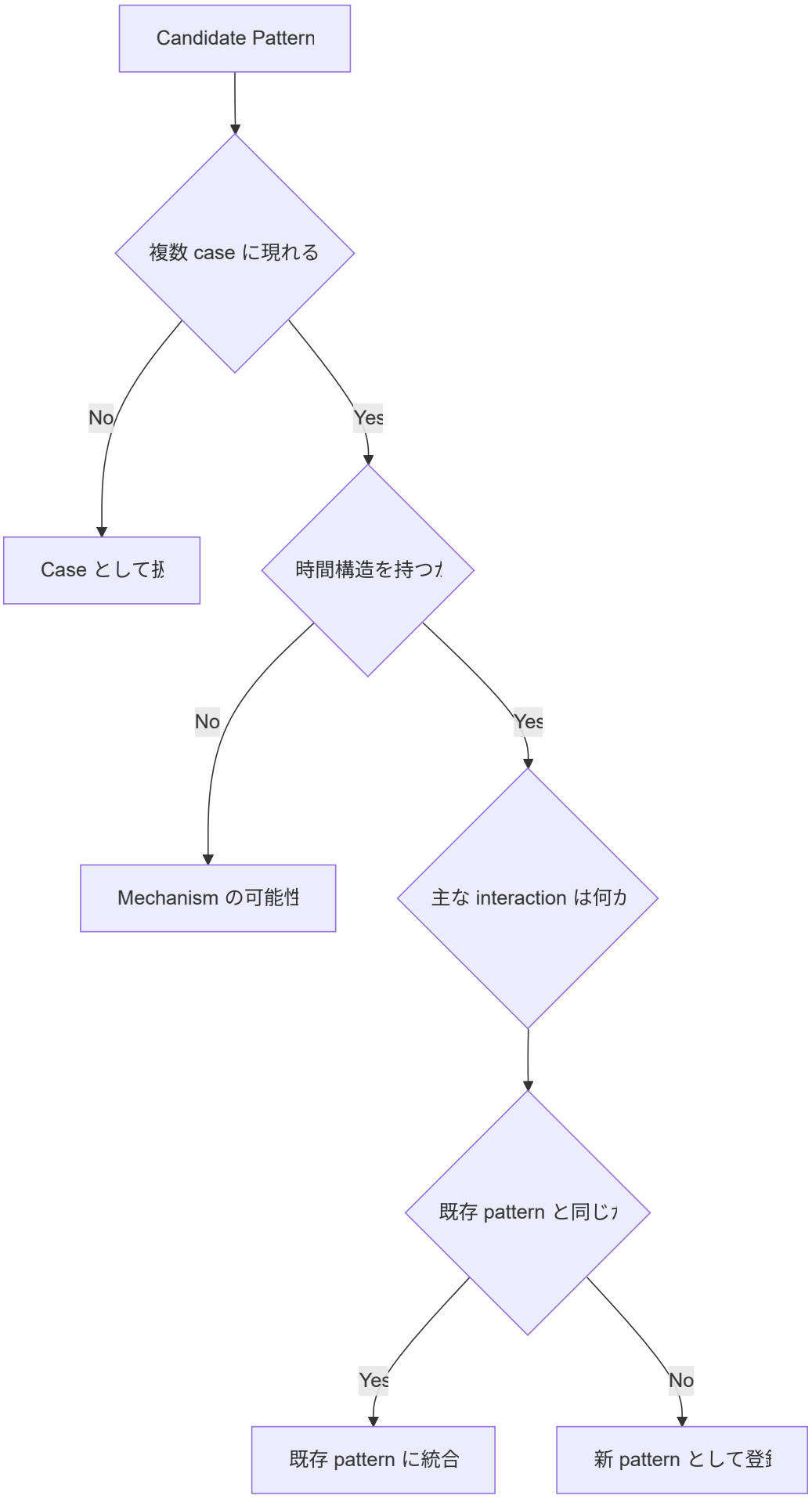
Pattern Boundary を決める方法

Pattern 境界は次の3つで決まる。

1. Trigger
2. Interaction
3. Outcome

この組み合わせが変わると
別 pattern になる。

Pattern 境界判定フロー


Candidate Pattern
複数 case に現れる
No
Yes
Case として扱
時間構造を持つか
No
Yes
Mechanism の可能性
主な interaction は何か
既存 pattern と同じか
Yes
No
既存 pattern に統合
新 pattern として登録

Pattern の境界を作る3つの軸

Pattern の境界は次の軸で区別できる。

---

1 Trigger の違い

同じ結果でも、
トリガーが違えば別 pattern。

例

- 規範逸脱トリガー
- 利益衝突トリガー
- 情報漏洩トリガー

---

2 Interaction の違い

同じトリガーでも
相互作用が違えば別 pattern。

例

- 同調拡散
- 対立激化
- 集団分裂

---

3 Outcome の違い

進行が同じでも
結果が違えば別 pattern。

例

- 炎上 → 沈静
- 炎上 → 社会制裁
- 炎上 → 文化変化

Pattern と Mechanism の違い

項目	Pattern	Mechanism
意味	起き方	原因
形式	進行	因果
時間	ある	ない場合もある
例	炎上パターン	同調メカニズム

Pattern と Structure の違い

項目	Pattern	Structure
意味	進行	配置
時間	ある	ない
例	権力争いパターン	階層構造

Pattern と Case の違い

項目	Pattern	Case
抽象度	高い	低い
再現性	ある	一度限り
例	規範逸脱炎上	具体事件

Pattern Boundary を守るためのルール

Rule1

pattern は **進行を持つこと**

Rule2

pattern は **最低3 case 以上**から抽象すること

Rule3

mechanism は pattern に入れない

Rule4

role は pattern にしない

Rule5

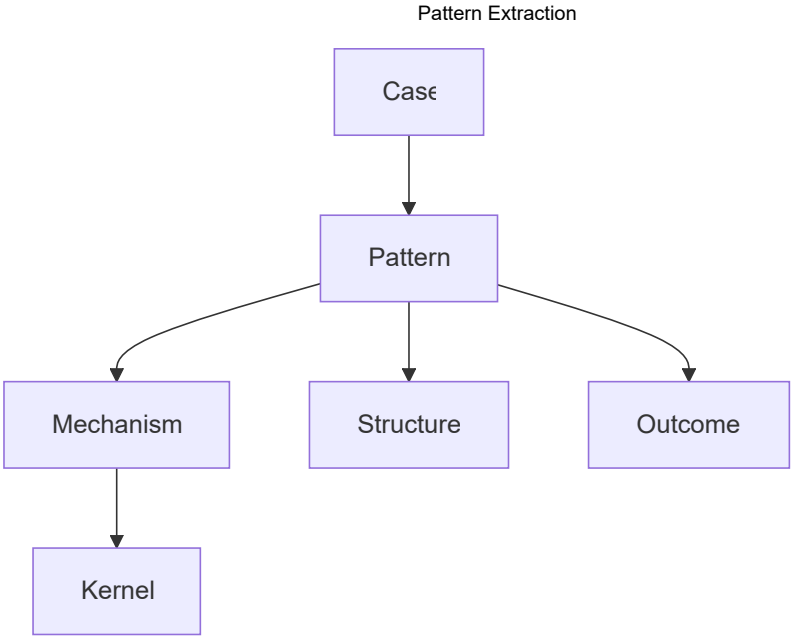
outcome は pattern 名にしない

Pattern Boundary のチェックリスト

新しい pattern を作る前に次を確認する。

- 複数 case に現れるか
- 時間進行があるか
- mechanism ではないか
- role ではないか
- outcome ではないか
- 既存 pattern に統合できないか

Pattern Boundary の図



Pattern Boundary を破る典型例

1 抽象レベル混乱

例

- 炎上パターン
- SNS炎上パターン
- 企業炎上パターン

これは階層化すべき。

---

2 mechanism 混入

例

- 同調パターン

同調は mechanism である。

---

3 outcome pattern

例

- 崩壊パターン

これは結果。

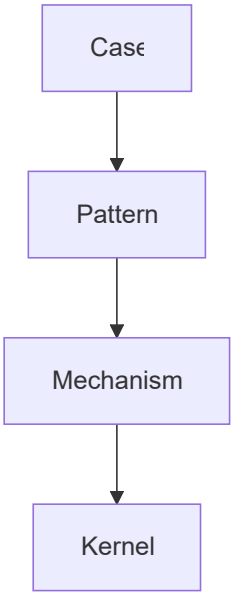
良い Pattern の条件

良い pattern は次を満たす。

- 複数 case に出る
- 進行が明確
- actor / role が分かる
- mechanism が説明できる
- outcome が複数ありうる

Knowledge Graph での位置

Pattern は Knowledge Graph の中間層に位置する。



LLM にとっての意味

Pattern Boundary が明確だと、

LLM は

- case を pattern に昇格できる
- pattern を mechanism に分解できる
- pattern 同士を比較できる

逆に境界が曖昧だと、

- 同じ現象を別名で語る
- mechanism と混同する
- reasoning が崩れる

この Vault での運用

この Vault では次を推奨する。

pattern は

- domain/pattern
- model/pattern

などに置く。

各 pattern には必ず

- 代表 case
- mechanism
- outcome

を書く。

関連ノート

- 99_oldzettelkasten/04_knowledge_graph/Pattern
- Representative Case Rule
- 99_oldzettelkasten/04_knowledge_graph/Mechanism
- Structure
- 99_oldzettelkasten/04_knowledge_graph/Knowledge Graph
- Anchor Case
- old_zettelkasten/pattern/Pattern Hub

Case to Pattern Promotion

Case to Pattern Promotion

Case to Pattern Promotion は、
複数の具体事例から反復する共通形を抽出し、
case を pattern へ昇格させるための判断規則である。

Knowledge Graph では、case は現実との接地面であり、
pattern は再発する型の認識装置である。
この二つがつながらないと、Vault は

- 事例の倉庫にはなるが
- 一般化が弱く
- 再発予測ができず
- domain 横断も起きにくい

という状態に陥る。

Case to Pattern Promotion は、
「この事例は単独事件として置くべきか、
それとも繰り返す型として pattern に上げるべきか」
を判断するための規則である。

定義

Case to Pattern Promotion とは、
複数の case に共通して現れる構造・順序・役割分担・帰結を整理し、
それらを **反復可能な現象の型** として pattern ノードに昇格させる操作である。

その目的は次の通りである。

1. case の山から共通形を抽出する
2. 一回きりの出来事と反復型を区別する
3. prediction と comparison を可能にする
4. domain を超える再利用性を高める
5. mechanism 抽出の足場を作る
6. LLM が類例探索しやすい構造を作る

なぜ必要か

Vault は運用していると、
最初は case がどんどん増える。
しかし pattern 昇格が起きないと、次のような問題が生じる。

1. case の散乱

似た事例がバラバラに置かれ、関連性が見えない。

2. 毎回ゼロから解釈

同じ種類の失敗や炎上でも、毎回一件ずつ手で考えることになる。

3. 再発予測不能

「また起きる型」として認識されないため、予防や設計に使えない。

4. mechanism 抽出不能

pattern がないと、背後 mechanism に上がりにくい。

5. Graph の具体偏重

case は増えるが abstract layer が痩せる。

Case to Pattern Promotion は、
Knowledge Graph を「出来事の記録帳」から
再発構造の地図 へ変えるために必要である。

全体構造

Promotion の本質

Promotion の本質は、
「似た例が複数あったからまとめる」ことではない。
重要なのは、
何が共通し、何が違ってもなお同じ型とみなせるか
を判断することである。

つまり pattern は単なる平均ではない。
pattern とは、

- 反復する順序
- 反復する役割配置
- 反復する緊張関係
- 反復する崩れ方
- 反復する帰結

を持つ「型」である。

---

pattern に昇格すべき case とは何か

以下の条件があるとき、case は pattern 昇格候補になる。

1. 類似 case が複数ある

最低2件でも候補にはなるが、3件以上あると強い。

2. 共通する順序がある

出来事の並びに反復性がある。

例:

- 逸脱
- 可視化
- 非難集中
- 境界確認
- 排除

3. 共通する役割がある

登場主体が違っても、役割配置が似る。

例:

- 逸脱者
- 規範維持者
- 煽動者
- 観衆
- 仲裁者

4. 共通する outcome がある

結末や帰結が一定方向へ寄る。

5. その抽象化が今後使える

pattern 化したときに、

- 比較
- 予測
- 設計
- domain 横断
  に役立つ。

Promotion Trigger

Promotion を始めるきっかけは次のようなものである。

---

1. Repeated Similar Cases

似た case が複数蓄積したとき。

例:

- 複数の炎上事例
- 複数の責任回避事例
- 複数の制度疲労事例

---

2. Shared Sequence

発生順序が似ているとき。

例:

- 期待形成
- 誤解
- 失望
- 攻撃集中
- 信頼崩壊

---

3. Shared Roles

登場人物は違うが役割配置が似ているとき。

例:

- 支配者 / 被支配者
- 正統化主体 / 反対主体
- 逸脱者 / 制裁者 / 観衆

---

4. Shared Outcome

帰結が繰り返し似るとき。

例:

- 排除
- 責任不明化
- 制度形骸化
- 信頼低下

Promotion Criteria

pattern に上げる前に確認すべき基準がある。

---

1. Repetition

本当に複数回現れているか。

確認すること:

- 少なくとも複数事例に現れるか
- 一件限りの特殊事情ではないか
- 別文脈でも見られるか

---

2. Structural Similarity

表面的要素ではなく、構造が似ているか。

たとえば:

- 人物名が違う
- 時代が違う
- media が違う

それでも

- 役割配置
- 力関係
- 進行順
- 破綻箇所
  が似ていれば、pattern 候補になる。

---

3. Difference Tolerance

何が違っても同じ pattern として扱えるかを確認する。

これは非常に重要である。

pattern は完全一致ではない。
違いがあっても同じ型として扱える必要がある。

例:

- 炎上の規模は違う
- 主体は個人か企業か違う
- 媒体は SNS かリアル集団か違う

しかし

- 規範逸脱の可視化
- 集団的制裁
- 境界確認
  が共通なら、同型とみなせる。

---

4. Abstraction Usefulness

pattern 化することで、何が良くなるか。

pattern に上げる価値があるのは、次のような場合。

- 新しい case を分類しやすくなる
- 他分野へ応用できる

- 背後 mechanism が見えやすくなる
- 予防や介入設計に使える

価値が薄いなら、無理に pattern 化しない。

---

5. Cross Context Portability

別 context にも移せるか。

例:

- fandom 炎上
- 宗教的異端排除
- 組織内スケープゴート形成

これらが同じ型なら、pattern の価値は高い。

Promotion 手順

Promotion は次の順で行うと安定する。

Step 1. Collect Cases

候補 case を集める。

条件:

- 似ていると感じる case
- outcome が近い case
- 順序が近い case
- role 配置が近い case

Step 2. Compare Cases

比較軸で並べる。

比較軸の例:

- 発端
- 主体
- role 配置
- 進行順
- 強制 / 説得 / 可視化
- outcome
- 再発条件

Step 3. Extract Common Core

共通核を取り出す。

ここで抽出するのは、

- 表面的な素材
  ではなく
- 反復する骨格
  である。

Step 4. Name Pattern

pattern 名を付ける。

命名のコツ:

- 抽象すぎない
- 一事例固有すぎない
- 作動や帰結が見える
- 他 case にも適用しやすい

悪い例:

- A社炎上型
- なんか攻撃されるやつ

良い例:

- 規範逸脱可視化パターン
- 責任拡散パターン
- 正統性崩壊パターン

Step 5. Link Pattern to Cases

新 pattern から各 case へ `observed_in` / `instance_of` 関係でつなぐ。

基本:

- Case → Pattern は `instance_of`
- Pattern → Case は `observed_in` 的に本文で示す

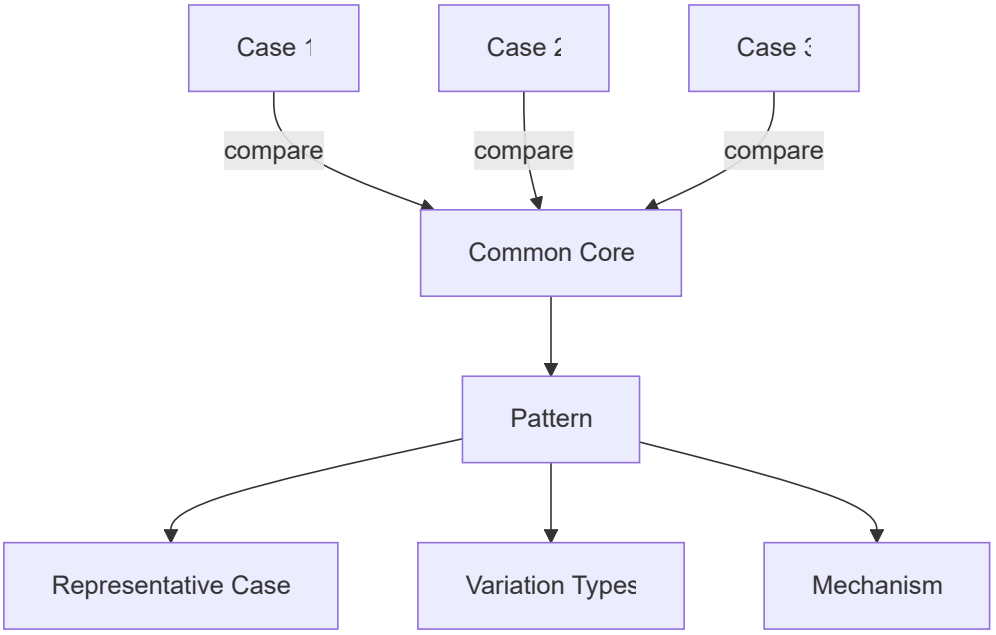
Step 6. Link Pattern to Mechanism

pattern の背後に mechanism があるなら接続する。

例:

- 責任回避パターン
  → 責任分散メカニズム
- 規範逸脱可視化パターン
  → 集団規範防衛メカニズム

Promotion の図



---

pattern ノートに書くべき内容

promotion 後の pattern ノートには最低限、次を書く。

1. 定義

この pattern は何か。

2. 発生条件

どんな条件で起きやすいか。

3. 典型的進行

どういう順序で進むか。

4. 主要 role

どんな役割が現れやすいか。

5. 典型的 outcome

どう終わりやすいか。

6. variation

どこが違っても同じ pattern とみなせるか。

7. representative case

代表 case は何か。

8. related mechanism

背後 mechanism は何か。

昇格してはいけないケース

次のようなものは、まだ pattern に上げない方がよい。

---

1. 一件限りで特殊性が強い

共通性より特殊事情が大きい。

2. 共通点が表層だけ

人物属性や時代背景だけ似ていて、構造が違う。

3. 名前だけ先走る

面白い名前は付けられるが、実体がない。

4. 比較対象が足りない

最低限の variation を見ていない。

5. pattern にしても使い道がない

単にまとめただけで、比較・予測・設計に役立たない。

よくある Promotion の失敗

1. Premature Abstraction

まだ2件程度で、しかも表面的類似だけなのに pattern 化する。

2. Overfitting to One Case

実質的に1件の説明を pattern 名にしただけになる。

3. Naming Bias

名前が強すぎて、他 case が見えなくなる。

例:

- 「〇〇型炎上」など固有名寄りすぎる名称

4. Ignoring Counterexamples

反例や違う系統を無視して pattern を固定する。

5. Mechanism と混同

pattern は「繰り返す型」であり、
mechanism は「それを生む作動過程」である。

---

pattern と mechanism の違い

これは Promotion で特に重要。

Pattern

- 繰り返し現れる現象の型
- 順序・役割・帰結の反復
- 観察寄り

Mechanism

- その型を生む作動過程
- 因果寄り
- 説明寄り

例:

- 責任回避パターン = Pattern
- 責任分散メカニズム = Mechanism

pattern 化の次に、必要なら mechanism 化を考える。

---

pattern の境界条件

良い pattern には、
「何を含み、何を含まないか」が必要である。

書くべきこと:

- この pattern の必須条件
- 任意条件
- 似ているが別 pattern のもの
- 適用しないケース

これがないと pattern が膨張する。

Promotion 判断のための質問

新しい case 群を見たとき、次の問いで判定する。

1. これは複数回現れているか

2. 共通する順序があるか

3. 共通する role 配置があるか

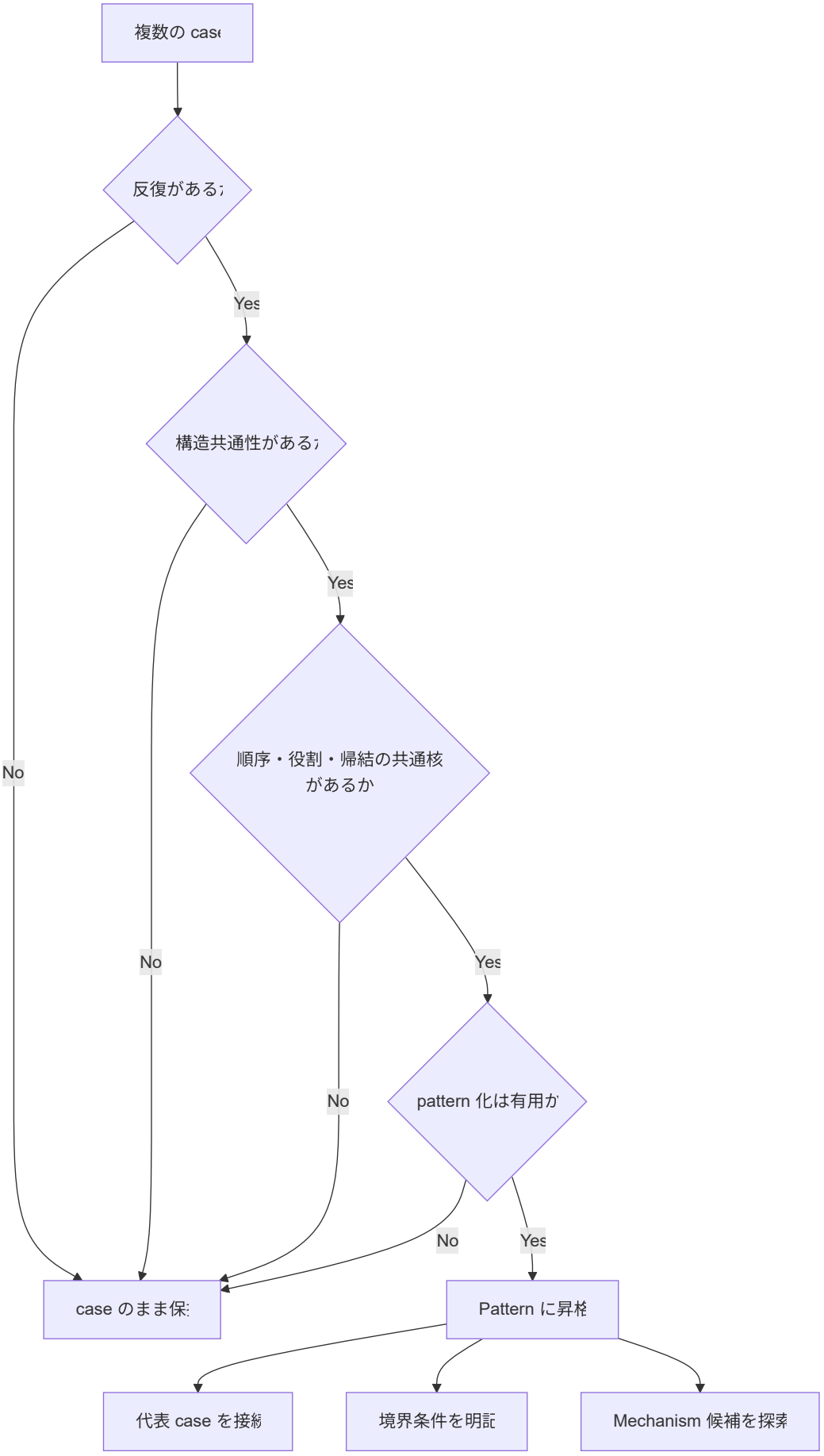
4. 結果が似ているか

5. 表層差を超える構造共通性があるか

6. pattern 化すると比較や予測に役立つか

7. 反例を見てもなお共通核が残るか

Promotion 判定フロー


複数の case
反復がある
Yes
No
構造共通性がある
Yes
No
順序・役割・帰結の共通核があるか
Yes
No
pattern 化は有用か
No
Yes
case のまま保
Pattern に昇格
代表 case を接続
境界条件を明記
Mechanism 候補を探索

LLM にとっての意味

Case to Pattern Promotion があると、LLM は

- 新しい case を既存 pattern に分類しやすくなり
- 類例探索が速くなり
- 単発の印象論でなく反復構造として説明しやすくなり
- 背後 mechanism への抽象化がしやすくなる

逆に promotion がないと、
LLM は毎回 case ごとにゼロから連想しやすい。

つまり Promotion は、
LLM に「この Vault における再発型の辞書」を与える。

この Vault における実装方針

この Vault では、case が3件以上似たら、
最低一度は pattern 昇格を検討する。

推奨運用:

- case を並べた比較メモを作る
- 共通核を抽出する
- 仮 pattern 名をつける
- 反例が出たら境界条件を修正する
- pattern が安定したら mechanism を探る

重要:

- 一気に mechanism へ飛ばない
- まず pattern を作る
- その後に背後 explanation を整える

他ノートとの接続

上位

- 99_oldzettelkasten/04_knowledge_graph/Knowledge Graph

近接

- 99_oldzettelkasten/04_knowledge_graph/Graph Maintenance
- 99_oldzettelkasten/04_knowledge_graph/Reasoning Path
- 99_oldzettelkasten/04_knowledge_graph/Traversal
- 99_oldzettelkasten/04_knowledge_graph/Bridge Concept
- Anchor Case

下位候補

- 02_zettelkasten/05_case_intelligence/03_pattern_extraction/Pattern Boundary Rule
- Representative Case Rule
- Pattern Naming Rule
- Pattern to Mechanism Promotion

まとめ

Case to Pattern Promotion は、
複数の具体事例から**反復する共通形**を抽出し、
case を pattern へ昇格させるための判断規則である。

これにより、

- case の散乱を防ぎ
- 比較と予測を可能にし
- mechanism 抽出の足場を作り
- domain 横断を強め
- LLM の類例推論を安定化できる

case が地面の石だとすれば、
pattern はそこに現れる地層の線である。